

Relatório 21 – Visão Computacional

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre Visão Computacional, utilizando a biblioteca PyTorch. A tarefa consistia em assistir a aulas do curso “*The Ultimate Computer Vision and Deep Learning Course*” e ao final, realizar uma atividade prática para demonstrar compreensão do conteúdo.

2. Desenvolvimento

Um das coisas que notei durante o curso é que eu sabia toda a parte teórica de pelo menos 5 das 7 seções, acredito que o que veio de aprendizagem foi a diferença de uma biblioteca para outra e como funciona por baixo dos panos, por exemplo, enquanto no TensorFlow você só chama a função `.fit()`, no PyTorch, o loop de treino é feito inteiramente na mão, `zero_grad`, `backward`, `step`, basicamente vira minha responsabilidade fazer a cada iteração.

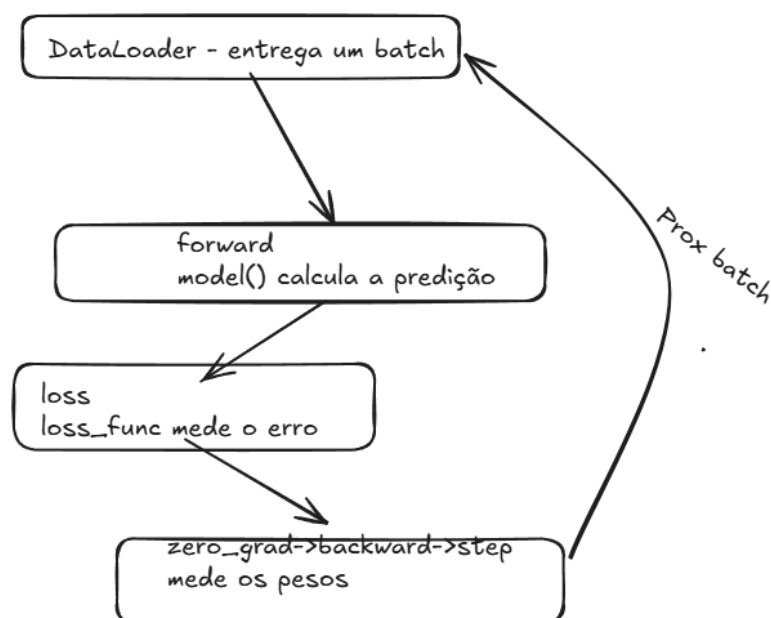


Figura 1. Ciclo de treino do PyTorch, feito manualmente. (autoria própria)

Outra coisa que notei, é que o gerenciamento de device é feito manualmente também, no TensorFlow nem havia notado que precisa configurar ele. A arquitetura pode simplesmente virar uma classe vez de só ir empilhando camada uma por cima da outra, o que aparenta mais programação orientada a objetos do que o Sequential do Keras. Meio que o PyTorch te obriga entender cada coisa para fazer, algo semelhante

a dois frameworks que usei em um projeto pessoal, LangGraph e Agno, o TensorFlow acaba sendo mais fácil de usar, bom para quem tem um conhecimento já e PyTorch para quem tá aprendendo como funcionar mais por trás dos panos.

Sobre a parte prática, eu iniciei fazendo um junção de várias partes de códigos da aula. O primeiro foi do notebook da seção 5, com 2 blocos (composto de Conv2d, MaxPool2d e ReLU) empilhados, o Dataset e DataLoader do torchvision e o ciclo de treino manual (zero_grad, backward e step) com a diferença que troquei o dataset para o CIFAR-100 (100 classes com 500 imagens de treino por classe, imagens com resolução de 32x32), usando transforms.RandCrop com transforms.RandomHorizontalFlip como data augmentation, através do transform do próprio torchvision.datasets).

Fiz mudanças que foram testadas de forma isolada para conseguir adicionar o efeito de cada uma, basicamente, o resumo delas é esse:

Teste	Configuração	Épocas	Pico de acurácia
1	2 blocos (conv, maxpool, relu)	10	45,1%
2	3 blocos (conv, maxpool, relu)	10	42,3%
3	3 blocos (conv, batchnorm, maxpool, relu)	25	56,4%
4	+ dropout de 0.5	45	58,0%
5	+ LR Scheduler (StepLR)	45	59,1%
6	Redução do dropout para 0.3	45	60,4%
7	+ ColorJitter	45	60,0% (não ajudou)
8	+ canais largos (128, 256, 512) + weight_decay	45	64,2%
9	Aumento do dropout para 0.4 + weight_decay maior	45	64,0%

Basicamente:

- 1 para o 2: aumentar um bloco de convolução sem mais nada piorou o resultado, meio que só profundidade não ajuda.
- 2 para o 3: maior salto dos testes, a convergência ficou mais rápida já nas primeiras épocas.
- 3 para 4: com 25 épocas, o dropout só piorou, mas era porque o modelo não convergiu por causa do ruído que o dropout coloca de propósito. Aumentando para 45 épocas, o dropout ajudou, sem ele, a perda na validação começava subir depois de época 16, com ele, a perda foi caindo até a última época.
- 4 para o 5: o scheduler acelerou a convergência sem abrir overffiting, deu para ver alguns saltos de acurácia logo depois de cada redução da taxa de aprendizado, nas épocas 16 e 31 onde o StepLR diminuía o lr pela metade.

- 5 para o 6: como não tava aparecendo overfitting, tentei testar com menos dropout, até funcionou já que houve um ganho pequeno.
- 6 para o 7: o ColorJitter não ajudou, provavelmente porque o dataset já é de baixa resolução e a cor é saturada, então, mexer na cor não ia ensinar nada de novo.
- 7 para o 8: foi o segundo maior salto, aqui eu só aumentei os canais (de 64, 128, 256 para 128, 256, 512) para ter mais capacidade de representação com mais regras L2 para não virar overfitting.
- 8 para o 9: o resultado não mudou muito, então parei aqui.

No último, eu havia aumentado o Dropout para 0.4 e o weight_decay para 3e-4 (ante tava 1e-4), mas não mudou o resultado, então eu concluo que era só uma limitação (talvez por eu estar até as 3 da manhã testando, pode ter impactado na decisão também) estrutural mesmo.

A estrutura final dela foi:

Layer (type)	Output Shape	Param #
Conv2d	[-1, 128, 30, 30]	3,584
BatchNorm2d	[-1, 128, 30, 30]	256
MaxPool2d	[-1, 128, 15, 15]	0
ReLU	[-1, 128, 15, 15]	0
Conv2d	[-1, 256, 13, 13]	295,168
BatchNorm2d	[-1, 256, 13, 13]	512
MaxPool2d	[-1, 256, 6, 6]	0
ReLU	[-1, 256, 6, 6]	0
Conv2d	[-1, 512, 4, 4]	1,180,160
BatchNorm2d	[-1, 512, 4, 4]	1,024
MaxPool2d	[-1, 512, 2, 2]	0
ReLU	[-1, 512, 2, 2]	0
Flatten	[-1, 2048]	0
Dropout	[-1, 2048]	0
Linear	[-1, 256]	524,544
ReLU	[-1, 256]	0
Linear	[-1, 100]	25,700
Total params:	2,030,948	
Trainable params:	2,030,948	
Non-trainable params:	0	

3. Conclusão

O conteúdo que mais utilizei foi o da seção 5 do curso, CNN + Data Augmentation, já que ele se mostrou bem útil para passar para um dataset mais difícil, sem precisar mudar nenhuma biblioteca e só ajustando a arquitetura e hiperparâmetros dentro do que já havia sido ensinado. Sobre o PyTorch, o conteúdo em si eu já tinha algum

conhecimento, porém, o curso ensinou mais sobre como funciona o treinamento de forma mais baixo nível, o que não é visível com o TensorFlow/Keras. Em relação a atividade prática, acredito que apenas peguei o código a aula e apliquei ele ao meu problema, no caso, o novo dataset.

4. Referências

The Ultimate Computer Vision and Deep Learning Course, disponível em:
<https://www.udemy.com/course/the-ultimate-computer-vision-and-deep-learning-course/>